

Cours 6 — Représentation des caractères · Interaction textuelle

Semaine 6 ■ Manuel Hachette ch. 9 (§ 9.4) et ch. 3 (§ 3.1) ■ 2 × 1 h

Objectifs

- Expliquer comment un texte est codé en machine (ASCII, Unicode, UTF-8)
- Utiliser `ord` et `chr`
- Écrire un programme interactif propre avec `input` et les f-strings

Partie 1 — Coder les caractères (cours A)

1.1 ASCII (1963)

Un texte est une suite de caractères, donc en machine... une suite de nombres. Le code **ASCII** associe un numéro sur **7 bits** (0 à 127) à chaque caractère de l'anglais : lettres, chiffres, ponctuation, caractères de contrôle.

Repères à connaître :

- `'A'` → 65, `'B'` → 66, ... `'Z'` → 90
- `'a'` → 97 (minuscule = majuscule + 32)
- `'0'` → 48 (le **caractère** `'0'` n'est pas le nombre 0!)
- espace → 32

1.2 Le problème des accents, et Unicode

ASCII n'a ni « é », ni « ç », ni idéogrammes, ni émojis. Chaque pays a d'abord créé sa propre extension à 8 bits (ISO-8859-1 « latin-1 » pour l'Europe de l'Ouest...), d'où des textes illisibles d'un pays à l'autre (les fameux Ã© à la place de é!).

Unicode attribue un numéro unique (*point de code*) à plus de 150 000 caractères de toutes les écritures : é = U+00E9, € = U+20AC, le smiley souriant = U+1F642.

1.3 UTF-8

UTF-8 est la façon la plus répandue d'écrire les numéros Unicode en octets :

- les caractères ASCII tiennent sur **1 octet** (compatibilité totale : un fichier ASCII est déjà de l'UTF-8);
- les autres caractères occupent 2, 3 ou 4 octets.

Donc : la **taille en octets** d'un texte $\neq$ son **nombre de caractères** (`"été"` : 3 caractères, 5 octets en UTF-8).

1.4 En Python

```
ord("A")      # 65      : caractère -> point de code
chr(233)       # 'é'     : point de code -> caractère
```

Partie 2 — Interaction textuelle (cours B)

2.1 Entrées et sorties

Un programme interactif suit le cycle : **saisie** → **traitement** → **affichage**.

```
reponse = input("Votre âge ? ")    # renvoie TOUJOURS une chaîne
age = int(reponse)                 # conversion explicite
```

Si la saisie n'est pas convertible, `int(...)` lève une `ValueError` (cf. semaine 4).

2.2 Affichage soigné : les f-strings

```
prenom = "Ada"
age = 16
print(f"{prenom} aura {age + 1} ans l'an prochain.")
print(f"Prix : {19.987:.2f} euros")    # arrondi : 19.99
```

Une f"`...`" insère la **valeur** des expressions entre accolades : plus lisible que les concaténations et les virgules dans `print`.

2.3 Structurer un dialogue : le menu

Le motif « boucle de menu », qu'on réutilisera tout l'an :

```
choix = ""
while choix != "q":
    print("(a) jouer (b) scores (q) quitter")
    choix = input("Choix ? ")
    if choix == "a":
        ...
```

À retenir

- Texte = suite de numéros ; ASCII (7 bits, anglais) $\subset$ Unicode (tout) ; UTF-8 = encodage dominant.
- `ord('A')` = 65, `chr(97)` = 'a' ; minuscule = majuscule + 32.
- `input` renvoie une chaîne : convertir avant de calculer.
- f-string : f"`texte {expression}`", format `:.2f` pour deux décimales.